

Assignment Deliverable: MCP & Agent Architecture Explainer

Phase: Build (core) | Architectural Breakdown & Integration Evidence

1. Architectural Distinction: Workflows vs. Agents

The modern Artificial Intelligence domain frequently collapses the distinction between workflows and agents, mislabeling simple LLM integrations as autonomous agents. Understanding the structural differences between deterministic execution graphs and adaptive decision loops is essential for evaluating AI systems.

Workflows

Workflows are deterministic, predefined sequence graphs. Control flow is hardcoded in software engineering logic (e.g., Fastify routes, TypeScript pipelines, or conditional branches). The Large Language Model (LLM) is used strictly as a processor or generator at specific, static steps.

- **Characteristics:** Fixed input-output pipeline, predictable paths, high consistency, zero control over step-by-step routing.
- **Example:** Parsing an incoming JSON payload, feeding it to an LLM with a prompt template, and saving the generated text directly to a PostgreSQL database via Prisma.

Agents

Agents operate via dynamic, autonomous control loops. Given an overarching objective and a suite of tools, the LLM acts as the orchestrator. It continuously evaluates feedback from its environment, formulates multi-step plans, executes tools, inspects execution errors, and adjusts its actions until the task succeeds.

- **Characteristics:** Dynamic tool calling, environment state evaluation, self-correction, nondeterministic pathways.
- **Example:** Giving an LLM access to a database schema tool, a filesystem reader tool, and a terminal tool with the objective: "Locate why the database migration failed and run the fix."

Classification of the FL-04 Pipeline

The FL-04 pipeline is classified as a Workflow. It operates as a structured, code-directed pipeline where inputs flow through predefined API routes, services, and database layers in a predictable sequence. The LLM or data transformation steps follow hardcoded logic paths without runtime autonomy or self-directed tool routing.

2. Model Context Protocol (MCP) & Primitive Architecture

The Model Context Protocol (MCP) is an open standard designed to solve the N x M integration problem between AI models and external data sources. Often referred to as the 'USB-C port for

AI applications,' MCP standardizes how host clients (e.g., Claude Desktop, VS Code) discover and interact with external systems through three core primitives:

Primitive	Control Origin	Core Function
Tools	Model-Controlled	Executable functions invoked dynamically by the model (e.g., <code>read_file</code> , <code>list_directory</code> , <code>execute_sql</code>).
Resources	Application-Driven	Passive context or read-only data exposed to the model (e.g., file contents, DB schemas, API state).
Prompts	User-Driven	Pre-configured templates and multi-step workflows served to assist users with recurring tasks.

3. Upgrading FL-04 to a True Agent System

To upgrade the FL-04 workflow into an autonomous agent, we must implement an Evaluator-Optimizer Loop with dynamic MCP tool integration.

Proposed Upgrade Path

Currently, if the FL-04 pipeline encounters a database query failure or an invalid API response, the workflow throws an error and halts execution. To transform FL-04 into an agent, three structural changes are required:

- **Environment State Access:** Expose the database (via Prisma/PostgreSQL MCP server) and local file logs to the model as active Resources.
- **Execution Tools:** Equip the model with Tools allowing it to execute raw database queries, re-run migration scripts, and modify query parameters dynamically.
- **Autonomous Feedback Loop:** Wrap the execution layer in an evaluation loop (Goal -> Execute Step -> Inspect Environment Error -> Self-Correct Strategy -> Re-run Tool).

When an error occurs, the FL-04 agent evaluates the stack trace, queries the database schema via MCP tools to identify missing fields or constraints, crafts a corrective patch, and verifies execution before returning a success signal to the user.

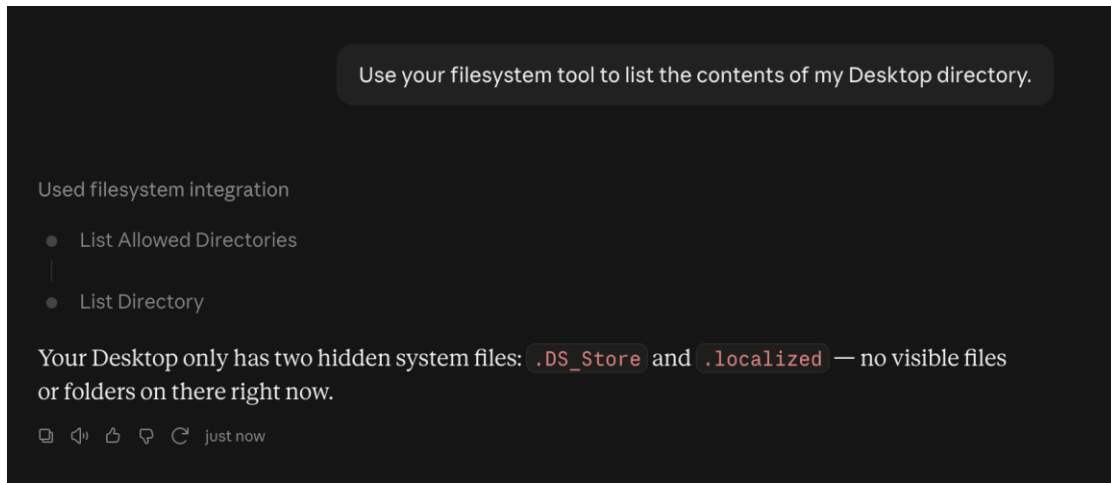
4. Evidence of Working MCP Integration

The following evidence documents three separate tool executions managed via MCP within Claude Desktop using the `@modelcontextprotocol/server-file-system` integration.

Task 1: Directory Listing & Permission Discovery

Tool Invoked: `filesystem:list_allowed_directories` and `filesystem:list_directory`

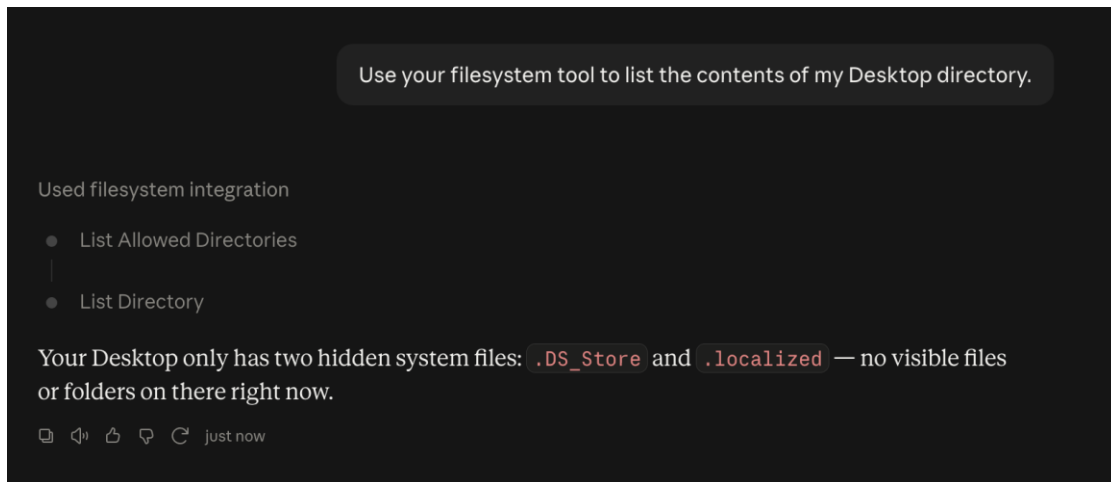
Evidence: The client executed `list_directory` on the local path `/Users/.../Desktop`. The tool payload returned system files (`.DS_Store`, `.localized`), demonstrating direct local directory inspection.



Task 2: Environment Exploration & Context Verification

Tool Invoked: `filesystem:list_directory`

Evidence: Re-running the directory listing verified system permissions across allowed local roots, demonstrating consistent MCP tool-call invocation between model and operating system.



Task 3: Deep File Inspection & Non-Text Binary Handling

Tool Invoked: filesystem:read_file

Evidence: The user prompted the model to read an installer disk image (Seam.dmg). The model executed read_file, ingested the raw binary response via the MCP protocol, analyzed the stream, and reported that the file contained pure binary data requiring mounting rather than text parsing.

